בס"ד

שמות חברי הצוות:

נאות דוכן

הודיה ארביב

גלי אהרון

תובנה 1

פורמט **Parquet** הציג זמני כתיבה נמוכים משמעותית בהשוואה ל־**CSV** בכל סדרי הגודל שנבדקו, כפי שנבדק בטבלת **lineitem** (הטבלה הגדולה ביותר מבין הטבלאות). עבור Scale Factor של 1.0, זמן הכתיבה של lineitem בפורמט **Parquet** (ללא דחיסה) היה כ־6.7 שניות, לעומת כ־15 שניות ב־**CSV**, כלומר פי כ־2.2 יותר איטי. דחיסת **snappy** שיפרה את הביצועים אף מעבר לכך, לכ־3.4 שניות בלבד. **Parquet**, ובפרט עם דחיסה, יעיל משמעותית יותר מבחינת ביצועי כתיבה, בפרט בטבלאות גדולות.

תובנה 2

ככל שגדל ה־**Scale Factor**, זמני הכתיבה גדלו בעקביות בטבלאות שנפחן תלוי ישירות בגודל סדר הגודל. בטבלת **lineitem** זמן הכתיבה בפורמט **Parquet** (ללא דחיסה) עלה מכ־3 שניות ב־SF=0.5 לכ־6.7 שניות ב־SF=1 וכ־24.4 שניות ב־SF=2, ומגמה דומה נראתה גם בקבצי **CSV** (מכ־9.1 לכ־15 לכ־60.3 שניות בהתאמה). בטבלאות קטנות וקבועות בגודלן, כגון nation ו־region, לא נראתה מגמת עלייה עקבית. זמני הכתיבה בהן נעו בטווח נמוך של כ־0.06 עד 0.2 שניות בכל הסקאלות, מושפעים בעיקר מעלויות קבועות של תזמון משימות ב־Spark ולא מנפח הנתונים עצמו. גידול הסקאלה משפיע ישירות על זמן הכתיבה בעיקר בטבלאות שגודלן אכן תלוי ב־Scale Factor

תובנה 3

שאילתת **countDistinct** הייתה בעקביות המהירה ביותר מבין השלוש, עם זמני ריצה של פחות מ־5 שניות בכל סדרי הגודל וכל הפורמטים שנבדקו. השאילתה **groupOrder** הייתה ברוב המקרים (8 מתוך 9 השילובים שנבדקו) האיטית ביותר, ולא **groupOrderJoin**, למרות שזו כוללת גם פעולת Join. ההסבר נעוץ בהערכה העצלה (lazy evaluation) של Spark: lineitem_df נשמר בזיכרון באמצעות **persist()**, אך ה־caching בפועל מתבצע רק כשמופעלת עליו הפעולה הראשונה בקוד, וזו **groupOrder**. לכן היא סופגת את עלות הקריאה מהדיסק וה־caching בפועל, בעוד ש־**groupOrderJoin** ו־**countDistinct**, הרצות אחריה, נהנות מנתונים שכבר נמצאים בזיכרון ולכן מהירות משמעותית יותר.